Verification-Driven Program Development
Exercise Sheet, Week 4

22.10.2026

Exercises

1. Fie următoarele definiții:

```
ghost predicate isEven(x : int)
{
  exists y : int :: x == y * 2
}

ghost predicate isEven'(x : int)
{
  x % 2 == 0
}

ghost predicate isOdd(x : int)
{
  exists y : int :: 2 * y + 1 == x
}

ghost predicate isOdd'(x : int)
{
  x % 2 == 1
}

ghost predicate isOdd''(x : int)
{
  !isEven(x)
}
```

O tehnică simplă de demonstrare este folosirea unei secvențe de assert-uri:

```
lemma odd_even_1()
  ensures forall n : nat :: isOdd(n) ==> isEven(n + 1)
{
  assert forall n: nat :: isOdd(n) ==> n % 2 == 1;
  assert forall n: nat :: n % 2 == 1 ==> (n + 1) % 2 == 0;
  assert forall n: nat :: n % 2 == 0 ==> n == 2 * (n / 2);
  assert forall n: nat :: n == (n / 2) * 2 ==> isEven(n);
}
```

Pornind de la modelul de mai sus, completați lema de mai jos:

```
lemma even_odd_1()
  ensures forall n : nat :: isEven(n + 1) ==> isOdd(n)
{
  assume false; // TODO
}
```

N.B. Înlocuiți `assume false;` cu demonstrația.

2. Considerăm următoarea variantă a lemei, în care n este dat ca parametru.

```
lemma odd_even_1'(n : int)
  ensures isOdd(n) ==> isEven(n + 1)
{
  assert isOdd(n) ==> n % 2 == 1;
  assert n % 2 == 1 ==> (n + 1) % 2 == 0;
  assert (n + 1) % 2 == 0 ==> (n + 1) == 2 * ((n + 1) / 2);
  assert (n + 1) == ((n + 1) / 2) * 2 ==> isEven(n + 1);
}
```

Pornind de la modelul de mai sus, completați lema de mai jos:

```
lemma even_odd_1'(n : int)
  ensures isEven(n + 1) ==> isOdd(n)
{
  assume false; // TODO
}
```

3. Considerăm următoarea variantă a lemei, în care n este dat ca parametru și ipoteza ca precondiție.

```
lemma odd_even_1''(n : int)
  requires isOdd(n)
  ensures isEven(n + 1)
{
  assert n % 2 == 1;
  assert (n + 1) % 2 == 0;
  assert (n + 1) == ((n + 1) / 2) * 2;
  assert isEven(n + 1);
}
```

Pornind de la modelul de mai sus, completați lema de mai jos:

```
lemma even_odd_1''(n : int)
  requires isEven(n + 1)
  ensures isOdd(n)
{
  assume false; // TODO
}
```

4. Iată un exemplu de folosire a tehnicilor de: (1) eliminare a cuantificatorului existențial și (2) de introducere a implicație și (3) introducere a cuantificatorului $\forall$:

```
lemma even_n_even_n_2(n : int)
  requires isEven(n)
  ensures isEven(n + 2)
{
  assert exists y : int :: n == y * 2;
  var half_n : int :| n == half_n * 2; // "exists elimination" rule in ND
  var half_n2 := half_n + 1;
  assert half_n2 * 2 == n + 2;
  assert exists y : int :: n + 2 == y * 2; // "exists introduction" rule in ND
                                    // the witness for y is half_n2
}

lemma even_n_even_n_2'(n : int)
  ensures isEven(n) ==> isEven(n + 2)
{
  if isEven(n) { // "implies introduction" rule in ND
    even_n_even_n_2(n); // call a lemma
  }
}

lemma even_n_even_n_2''()
  ensures forall n :: isEven(n) ==> isEven(n + 2)
{
  forall n : int // forall statement ("forall introduction" rule in ND)
    ensures isEven(n) ==> isEven(n + 2)
  {
    even_n_even_n_2'(n);
  }
}
```

Completați următoarele demonstrații:

```
lemma odd_n_odd_n_2(n : int)
  requires isOdd(n)
  ensures isOdd(n + 2)
{
  assume false; // TODO
}

lemma odd_n_odd_n_2'(n : int)
  ensures isOdd(n) ==> isOdd(n + 2)
{
  assume false; // TODO
}

lemma odd_n_odd_n_2''(n : int)
  ensures forall n : int :: isOdd(n) ==> isOdd(n + 2)
{
  assume false; // TODO
}
```

5. Iată un exemplu de folosire a tehnicii *Proof by contradiction*:

```
lemma examplePBC(n : nat) // in Romanian: demonstratie prin reducere la absurd
```

```
  requires isOdd(n)
  ensures !isEven(n * n)
{
  if isEven(n * n) { // assume n square were even
    var half_nsquare : nat :| n * n == half_nsquare * 2; // then find its half
    var half_n : nat :| n == 2 * half_n + 1;
    assert (2 * half_n + 1) * (2 * half_n + 1) == half_nsquare * 2;
    assert (2 * half_n + 1) * (2 * half_n + 1) == 4 * half_n * half_n + 2 * half_n + 2 * half_n
    assert (2 * half_n + 1) * (2 * half_n + 1) == 2 * (2 * half_n * half_n + half_n + half_n) +
    assert false; // derive a contradiction
  }
}

lemma examplePBC'(n : nat)
  requires isOdd(n)
  ensures !isEven(n * n)
{
  if isEven(n * n) {
    var half_nsquare : nat :| n * n == half_nsquare * 2;
    var half_n : nat :| n == 2 * half_n + 1;
    calc // calculational proof
    {
      half_nsquare * 2;
        ==
      n * n;
        ==
      (2 * half_n + 1) * (2 * half_n + 1);
        ==
      4 * half_n * half_n + 2 * half_n + 2 * half_n + 1;
        ==
      2 * (2 * half_n * half_n + half_n + half_n) + 1;
    }
    assert false;
  }
}
```

Folosind această tehnică, completați următoarea demonstrație:

```
lemma exercisePBC(n : nat)
  requires isEven(n)
  ensures !isOdd(n * n)
{
  assume false; // TODO
}
```

6. Iată un exemplu de folosire a tehnicii *case analysis*:

```
lemma exampleCaseAnalysis(n : nat)
  ensures isEven(n) || isOdd(n)
{
  if n % 2 == 0 {
    assert n == (n / 2) * 2;
  } else {
```

```
    assert n % 2 == 1;
    assert n == 2 * (n / 2) + 1;
  }
}
```

Folosind această tehnică, completați următoarea lemă:

```
lemma exampleCaseAnalysis'(n : nat)
  ensures isEven(n) || isEven(n + 1)
{
  assume false; // TODO
}
```

7. Iată un exemplu de demonstrație prin inducție:

```
function sum(n : int) : int
  requires n >= 0
{
  if n == 0 then
    0
  else
    n + sum(n - 1)
}

lemma {:induction false} exampleInduction(n : int)
  requires n >= 0
  ensures sum(n) == n * (n + 1) / 2
{
  if n == 0 {
    // nothing to do here in the base case
    // 0 == sum(0) == 0 * (0 + 1) / 2
  } else {
    calc {
      sum(n);
        ==
      n + sum(n - 1);
        ==
      {
        exampleInduction(n - 1);
      }
      n + (n - 1) * n / 2;
        ==
      n * (2 + (n - 1)) / 2;
        ==
      n * (n + 1) / 2;
    }
  }
}
```

Folosind aceeași metodă ca mai sus, completați următoarea lemă:

```
function sumSquares(n : int) : int
  requires n >= 0
```

```
{
  if n == 0 then
    0
  else
    n * n + sum(n - 1)
}

lemma {:induction false} exerciseInduction(n : int)
  requires n >= 0
  ensures sumSquares(n) == n * (n + 1) * (2 * n + 1) / 6
{
  assume false; // TODO
}
```

8. Completați lema ajutătoare de mai jos (indicație: folosiți inducție, ca în curs).

```
ghost predicate sorted1(s : seq<int>)
{
  forall i, j :: 0 <= i < j < |s| ==> s[i] < s[j]
}

ghost predicate sorted2(s : seq<int>)
{
  forall i :: 0 <= i < |s| - 1 ==> s[i] < s[i + 1]
}

lemma sorted1_sorted2(s : seq<int>)
  requires sorted1(s)
  ensures sorted2(s)
{
  forall i | 0 <= i < |s| - 1 // not necessary
    ensures s[i] < s[i + 1]
  {
    assert s[i] < s[i + 1]; //  instantiate i by i, j by i + 1 in forall i, j :: 0 <= i < j < |
  }
}

lemma helper(s : seq<int>, i : int, j : int)
  requires sorted2(s)
  requires 0 <= i < j < |s|
  ensures s[i] < s[j]
{
  if j == i + 1
  {
  }
  else
  {
    assume false; // TODO
    // HINT: use induction
  }
}

lemma sorted2_sorted1(s : seq<int>)
  requires sorted2(s)
```

```
  ensures sorted1(s)
{
  forall i | 0 <= i
    ensures forall j :: i < j < |s| ==> s[i] < s[j]
  {
    forall j | i < j < |s|
      ensures s[i] < s[j]
    {
      helper(s, i, j);
    }
  }
}
```

9. Alegeți tehnica de demonstrare adecvată pentru demonstrarea următoarelor leme:

```
  lemma even_even'_universal()
  ensures forall n :: isEven(n) <==> isEven'(n)
{
  assume false; // TODO
}

lemma odd_odd'_universal()
  ensures forall n :: isOdd(n) <==> isOdd'(n)
{
  assume false; // TODO
}

lemma odd_odd''_universal()
  ensures forall n :: isOdd(n) <==> isOdd''(n)
{
  assume false; // TODO
}

lemma even_square_even(n : nat)
  requires isEven(n * n)
  ensures isEven(n)
{
  assume false; // TODO
}

lemma odd_square_odd(n : nat)
  requires isOdd(n * n)
  ensures isOdd(n)
{
  assume false; // TODO
}

lemma even_n_times_n_1(n : nat)
  ensures isEven(n * (n + 1))
{
  assume false; // TODO
}

lemma even_mult(n : nat, m : nat)
```

```
  requires isEven(n * m)
  ensures isEven(n) || isEven(m)
{
  assume false; // TODO
}

lemma odd_mult(n : nat, m : nat)
  requires isOdd(n * m)
  ensures isOdd(n) && isOdd(m)
{
  assume false; // TODO
}
```